

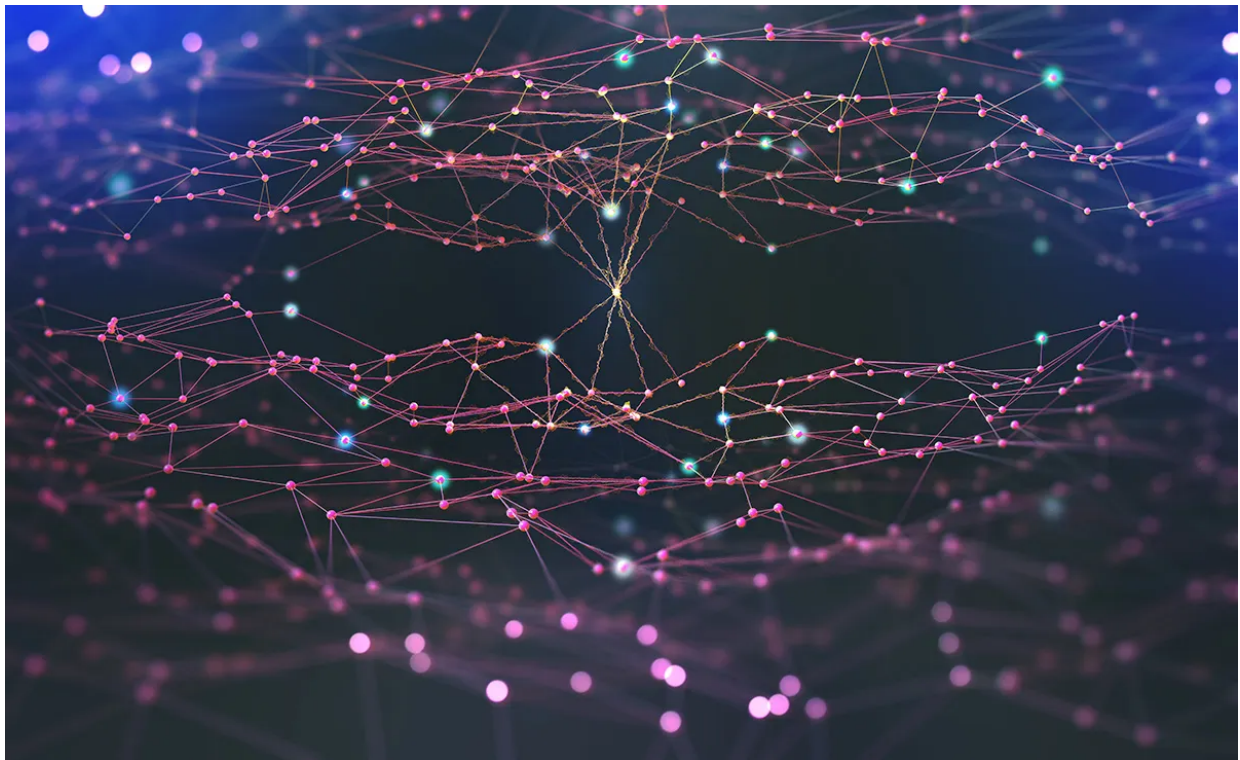
Benchmarking the MAX-CUT problem on the Simulated Bifurcation Machine



Yoshiki Matsuda

Follow

6 min read · Sep 30, 2019



Abstract

Using the Simulated Bifurcation Machine (SBM) published by Toshiba Digital Solutions Corporation on the AWS Marketplace, we have performed benchmarks of the combinatorial optimization problem MAX-CUT.

The Simulated Bifurcation Machine

SBM is a solver for combinatorial optimization problems provided by Toshiba Digital Solutions Corporation. By taking methods from classical physics used to describe bifurcation phenomena, adiabatic processes and Ergodic (or later chaos) theory and applying them to combinatorial optimization, SBM solves large problems quickly.

A proof-of-concept version is available on the AWS marketplace.

The MAX-CUT problem

We chose the MAX-CUT problem as a representative combinatorial problem that the SBM can solve.

The objective of the MAX-CUT problem is to split a graph into two mutually exclusive subgraphs such that the cumulative weight of inter-subgraph edges is maximal.

Objective function

Consider the case where we split graph V into the subgraphs S and T .

We define the binary value s_i , also called Ising variable, to express which subgraph vertex $v_i \in V$ is in as follows

$$s_i = \begin{cases} +1 & (v_i \in S) \\ -1 & (v_i \in T) \end{cases}$$

Using this variable, we define the objective function of the optimization problem as follows

$$\max H = \frac{1}{2} \sum_{1 \leq i < j \leq n} W_{ij} (1 - s_i s_j)$$

where W_{ij} is the weight of the edge between vertices v_i and v_j . Only in the

case where v_i, v_j are in opposing subgraphs is

$$(1 - s_i s_j) = 2.$$

When the two vertices are in the same subgraph, the weight of the edge connecting them will be multiplied by 0. Thus, maximizing this function will yield the solution to the MAX-CUT problem.

In annealing machines, the objective function is commonly minimized, not maximized. Multiplying the objective function by -1 lets us treat this as a minimization problem.

$$\min H = -\frac{1}{2} \sum_{1 \leq i < j \leq n} W_{ij} (1 - s_i s_j)$$

Additionally, defining a true binary

$$q_i = \frac{s_i + 1}{2} \in \{0, 1\}$$

Based on the Ising variable, we can rewrite the objective function as follows

$$\begin{aligned} \min H &= -\frac{1}{2} \sum_{1 \leq i < j \leq n} W_{ij} (1 - (2q_i - 1)(2q_j - 1)) \\ &= \sum_{1 \leq i < j \leq n} W_{ij} (2q_i q_j - q_i - q_j) \end{aligned}$$

This function can be used on annealing machines that expect true binaries like q_i .

Executing the SBM

MAX-CUT mode

The SBM includes a specialized MAX-CUT solver (MAX-CUT mode) that takes text input describing the problem. For the text input, the SBM supports the MatrixMarket format as well as that of the MAX-CUT benchmark dataset Gset.

The structure of the Gset format, which uses white space as separation for a list of integer values, is shown below

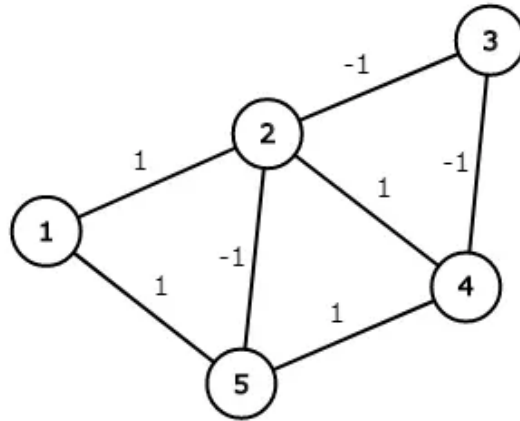
```
N E
i0 j0 w_i0j0
i1 j1 w_i1j1
i2 j2 w_i2j2
```

The first row lists the number of vertices N and edges E , while following rows list the edges one edge per row. Each such row contains the edge's endpoint vertices i and j as well as its weight w_{ij} .

As an example, the following shows a MAX-CUT problem with 5 vertices and 7 edges in Gset format

```
5 7
1 2 1
1 5 1
2 3 -1
2 4 1
2 5 -1
3 4 -1
4 5 1
```

Visualizing this as a graph results in the following figure.



The SBM parameters

The following parameters can be posted to the SBM.

- `steps` : The number of steps in the SBM calculation. The default is 0, in which case the number of steps is dynamically determined.
- `loops` : How many times the SBM calculation will be repeated to find better solutions. The default is 1, and a value of 0 will have the SBM run until the timeout time.
- `timeout` : The maximum calculation time in seconds. If executing the next loop would likely cause the SBM to overstep the timeout, calculations will be terminated early.
- `target` (optional): When a value is specified, execution will terminate when the objective function reaches said value.
- `stats` : The detail level of the statistical output returned. Statistics are collected over the `loops*steps` executions. Possible values are
 - `none` : Return no statistical output.
 - `summary` : Return average, standard deviation, and best frequency.
 - `full` : Return average, standard deviation, and a histogram.

Executing the example with five vertices from above-using curl looks as follows

```
$ echo -e "5 7\n1 2 1\n1 5 1\n2 3 -1\n2 4 1\n2 5 -1\n3 4 -1\n4 5 1" |  
curl -i -H "Content-Type: application/octet-stream" -X POST  
"http://192.168.0.1:8000/solver/maxcut?  
steps=0&loops=0&timeout=1&stats=full" --data-binary @-
```

Note that the IP-address (here `192.168.0.1`) varies based on your environment.

Get Yoshiki Matsuda's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

☒ Remember me for faster sign in

We are using the parameters listed below

- `steps = 0`
- `loops = 0`
- `timeout = 1`
- `stats = full`

If execution is successful, the SBM will send the following response.

```
HTTP/1.1 200 OK  
Content-Type: application/json; charset=utf-8  
Content-Length: 233  
ETag: W/"e9-6jYjTGcvinFBL6/eKnutYNzItOQ"  
Date: Mon, 19 Aug 2019 03:59:29 GMT  
Connection: keep-alive  
  
{  
  "id": "r1477108799",  
  "time": 1,  
  "wait": 0,  
  "runs": 396800,  
  "steps": 10,  
  "message": "timeout",  
  "value": 3,  
  "result": [0, 1, 0, 0, 1],  
  "stats": {  
    "avg": 2.441142,  
    "stddev": 1.372172,  
    "histogram": [[-2, 24009], [-1, 4124],  
                  [0, 15682], [1, 12766], [2, 12636], [3, 327583]]  
  }  
}
```

The central parts of the JSON response are the following

- `result` : The best solution found
- `value` : Value of the objective function for the best solution
- `runs` : Amount of executions (loops*steps)

Benchmarking with Gset

We will now execute the MAX-CUT benchmark dataset Gset, which can be downloaded from <http://web.stanford.edu/~yyye/yyye/Gset/>.

Gset consists of the problems G1 to G81 and, due to some skipped numbers, has a total of 71 problems with a graph size ranging from 800 to 20,000 vertices. There are graphs without weighted edges (all weights are 1) as well as graphs with weighted edges where the weights are either +1 or -1.

Geometrically, the structure of the graphs can be split into three categories

- Random graphs
- Planar graphs
- Toroidal graphs

Execution commands

For ease of use, we employ `curl` to post the Gset dataset to the SBM. For example, to execute problem G1 with the parameters `steps=0`, `loops=0`, `timeout=10` and `target=11624`, navigate to the directory where the Gset dataset is stored and execute the following command

```
$ cat G1 | curl -i -H "Content-Type: application/octet-stream" -X
POST "http://192.168.0.1:8000/solver/maxcut?
steps=0&loops=0&timeout=10&target=11624" --data-binary @-
```

Benchmark results

The following lists the results for 69 graphs in Gset. All executions were done with the parameters

`steps=0` (automatic), `loops=0` (automatic), `timeout=10` and `target=(best-`

known) , where best-known refers to the best-known values of the objective function for the given problem, which were taken from previous research.

instance	node	edge	weight	type	target	time[sec]	result	diff
G1	800	19176	+1	random	11624	0.02	11624	0
G2	800	19176	+1	random	11620	0.13	11620	0
G3	800	19176	+1	random	11622	0.09	11622	0
G4	800	19176	+1	random	11622	0.09	11622	0

Medium

 Search

 Get app

 Write

Sign up

Sign in



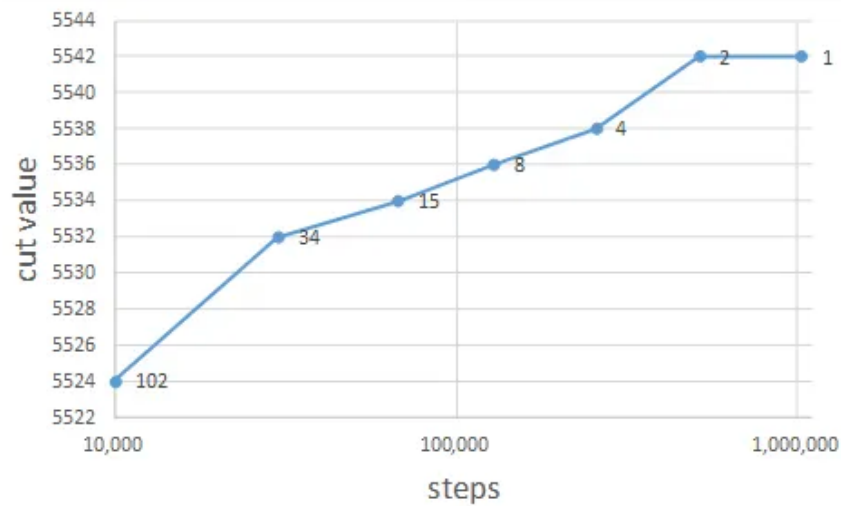
G6	800	19176	+1,-1	random	2178	0.02	2178	0
G7	800	19176	+1,-1	random	2006	0.02	2006	0
G8	800	19176	+1,-1	random	2005	0.02	2005	0
G9	800	19176	+1,-1	random	2054	0.02	2054	0
G10	800	19176	+1,-1	random	2000	0.06	2000	0
G11	800	1600	+1,-1	toroidal	564	0.01	564	0
G12	800	1600	+1,-1	toroidal	556	0.02	556	0
G13	800	1600	+1,-1	toroidal	582	0.02	582	0
G14	800	4694	+1	planar	3064	10.00	3063	1
G15	800	4661	+1	planar	3050	0.06	3050	0
G16	800	4672	+1	planar	3052	0.18	3052	0
G17	800	4667	+1	planar	3047	4.64	3047	0
G18	800	4694	+1,-1	planar	992	0.16	992	0
G19	800	4661	+1,-1	planar	906	0.02	906	0
G20	800	4672	+1,-1	planar	941	0.01	941	0
G21	800	4667	+1,-1	planar	931	0.08	931	0
G22	2000	19990	+1	random	13359	0.20	13359	0
G23	2000	19990	+1	random	13344	9.99	13342	2
G24	2000	19990	+1	random	13337	0.07	13337	0

G25	2000	19990	+1	random	13340	0.45	13340	0
G26	2000	19990	+1	random	13328	0.07	13328	0
G27	2000	19990	+1,-1	random	3341	0.07	3341	0
G28	2000	19990	+1,-1	random	3298	0.20	3298	0
G29	2000	19990	+1,-1	random	3405	0.07	3405	0
G30	2000	19990	+1,-1	random	3413	0.20	3413	0
G31	2000	19990	+1,-1	random	3310	0.79	3310	0
G32	2000	4000	+1,-1	toroidal	1410	2.08	1410	0
G33	2000	4000	+1,-1	toroidal	1382	9.77	1382	0
G34	2000	4000	+1,-1	toroidal	1384	0.78	1384	0
G35	2000	11778	+1	planar	7687	10.00	7680	7
G36	2000	11766	+1	planar	7680	10.01	7675	5
G37	2000	11785	+1	planar	7691	10.00	7685	6
G38	2000	11779	+1	planar	7688	10.00	7686	2
G39	2000	11778	+1,-1	planar	2408	10.00	2407	1
G40	2000	11766	+1,-1	planar	2400	4.95	2400	0
G41	2000	11785	+1,-1	planar	2405	10.00	2404	1
G42	2000	11779	+1,-1	planar	2481	9.87	2475	6
G43	1000	9990	+1	random	6660	0.02	6660	0
G44	1000	9990	+1	random	6650	0.02	6650	0
G45	1000	9990	+1	random	6654	0.02	6654	0
G46	1000	9990	+1	random	6649	0.02	6649	0
G47	1000	9990	+1	random	6657	0.02	6657	0
G48	3000	6000	+1,-1	toroidal	6000	0.07	6000	0
G49	3000	6000	+1,-1	toroidal	6000	0.07	6000	0
G50	3000	6000	+1,-1	toroidal	5880	0.07	5880	0
G51	1000	5909	+1	planar	3848	0.94	3848	0
G52	1000	5916	+1	planar	3851	1.77	3851	0

G53	1000	5914	+1	planar	3850	10.00	3849	1
G54	1000	5916	+1	planar	3852	10.00	3851	1
G55	5000	12498	+1	random	10299	10.00	10289	10
G56	5000	12498	+1,-1	random	4017	9.80	4008	9
G57	5000	10000	+1,-1	toroidal	3494	10.00	3480	14
G58	5000	29570	+1	planar	19293	10.01	19257	36
G59	5000	29570	+1,-1	planar	6086	10.00	6067	19
G60	7000	17148	+1	random	14188	10.00	14168	20
G61	7000	17148	+1,-1	random	5796	10.01	5777	19
G62	7000	14000	+1,-1	toroidal	4870	10.01	4844	26
G63	7000	41459	+1	planar	27045	10.01	26986	59
G64	7000	41459	+1,-1	planar	8751	8.82	8728	23
G65	8000	16000	+1,-1	toroidal	5562	10.01	5532	30
G66	9000	18000	+1,-1	toroidal	6364	9.01	6324	40
G67	10000	20000	+1,-1	toroidal	6950	10.01	6906	44
G70	10000	9999	+1	random	9591	10	9522	69
G72	10000	20000	+1,-1	toroidal	7006	10	6966	40

For smaller graphs, the preset optimization target was reached in less than one second in most cases.

For the above benchmark, we have let the system determine the appropriate number of `steps` automatically. However, we suspect that higher performance can be achieved by specifying the amount of `steps` a priori. To test this hypothesis, we have executed problem G65 with a time restriction of 60 seconds and the number of `steps` per loop between 10,000-1,000,000. The below figure shows the resulting solution for problem G65.



The numbers written next to data points indicate the number of loops that the SBM was able to execute within the time limit of 60 seconds.

For problem G65, setting the amount of `steps` to values greater than 500,000 results in more optimal solutions than running many loops with less `steps`.

Whether a large number of short loops or a small number of long loops is preferable seems to depend on the problem at hand. Thus, finding an appropriate number for `steps` is key for efficient solving.

Conclusion

Using the SBM, we were able to find approximate solutions to MAX-CUT problems at high speed. In particular comparatively small problems with about 2000 vertices were largely solved in less than a second.

Among the Ising formulated problems, MAX-CUT problems are considered relatively easy to solve since they have no constraints. While we have treated MAX-CUT problems from the Gset dataset for this benchmark, we intend to do similar tests with more complicated and realistic optimization problems. Through further tuning, we will also investigate how much more speed and precision can be achieved with the SBM.



Published in Toshiba SBM

28 followers · Last published Jan 6, 2020

Follow

Originated from research on quantum bifurcation machines, the SBM is a practical and ready-to-use ISING machine that solves large-scale “combinatorial optimization problems” at high speed.



Written by Yoshiki Matsuda

7 followers · 1 following

Follow

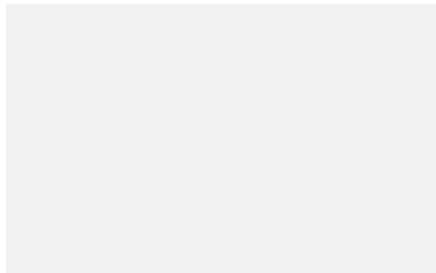
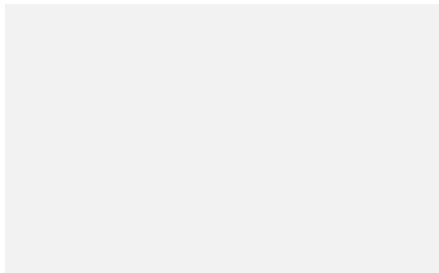


Placeholder for text content.

Placeholder for text content.

Placeholder for text content.

Placeholder for text content.



Placeholder for text content.

